

EXPERIMENT – 1

PROBLEM STATEMENT: Create a menu-driven application that performs the following Natural Language Processing tasks using the pipeline() function of the Transformers library:

- 1.Sentiment Analysis
- 2.Text Generation
- 3.Summarization
- 4.Question Answering
- 5.Translation

OBJECTIVES

- 1.To understand the use of the Hugging Face Transformers pipeline() function.
- 2.To perform different NLP tasks using pre-trained Transformer models.
- 3.To implement a menu-driven NLP application.
- 4.To understand automatic model selection using pipeline().
- 5.To understand how specific models can be selected using the model parameter.
- 6.To compare the behavior of different models for the same NLP task.
- 7.To observe the behavior of a model when it is used for a task for which it was not designed.
- 8.To understand the use of max_new_tokens and temperature during text generation.

THEORY

1. Overview of the Experiment

The Transformers library provides pre-trained machine learning models for various Natural Language Processing tasks. The pipeline() function provides a simple interface for using these models without implementing the complete model architecture manually.

In this experiment, a menu-driven application is developed that allows the user to select different NLP operations.

The application performs:

Sentiment Analysis

Sentiment Analysis determines the sentiment expressed in a given text, such as **positive** or **negative**.

Example:

Input: I really enjoyed this movie.

Output: POSITIVE

Text Generation

Text Generation generates new text based on a given prompt. In this experiment, the GPT-2 model is used.

The max_new_tokens parameter controls the maximum number of new tokens generated, while temperature controls the randomness of the generated text.

Summarization

Summarization generates a shorter version of a longer input text while retaining important information.

Question Answering

Question Answering extracts an answer from a given context according to the question provided by the user.

Translation

Translation converts text from one language to another. In this experiment, English text is translated into Hindi.

2. Libraries Used**Transformers**

The transformers library is used to access pre-trained Transformer models and the pipeline() function.

```
from transformers import pipeline
```

PyTorch

PyTorch is required by the Transformers models for model execution.

```
torch
```

SentencePiece

SentencePiece is used by several Transformer models for tokenization.

```
sentencepiece
```

The required packages can be installed using:

```
!pip install transformers==4.53.0
```

```
!pip install torch
```

```
!pip install sentencepiece
```

3. Functions Used**pipeline()**

Creates an NLP pipeline for a particular task.

Example:

```
pipeline("sentiment-analysis")
```

input()

Takes input from the user.

```
text = input("Enter text: ")
```


print()

Displays the result on the screen.

classifier()

Used to perform sentiment analysis after creating a sentiment-analysis pipeline.

generator()

Used to generate text from a given prompt.

summarizer()

Used to summarize input text.

qa()

Used to answer a question from a given context.

translator()

Used to translate text from one language to another

WORKING OF PROGRAM

The program starts by importing the pipeline function from the Transformers library.

A separate function is created for each NLP task:

- 1 .sentiment_analysis()
- 2 .text_generation()
- 3 .summarization()
- 4 .question_answering()
- 5 .translation()

The user is presented with a menu.

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Different Models
7. Model for Another Task
8. Exit

The selected option calls the corresponding function.

A. Pipeline Without Specifying Model

For example:

```
classifier = pipeline("sentiment-analysis")
```


Here, no model is explicitly specified. The pipeline automatically selects and downloads a suitable model.

The same approach is used for:

```
pipeline("summarization")
```

and:

```
pipeline("question-answering")
```

B. Using Specific Models

A model can also be explicitly specified using the model parameter.

For example:

```
translator = pipeline(  
    "translation",  
    model="Helsinki-NLP/opus-mt-en-hi"  
)
```

Different sentiment-analysis models can also be tested to compare their results.

C. Using a Model Intended for Another Task

The experiment also tests a model with a task for which it was not designed.

For example:

```
classifier = pipeline(  
    "sentiment-analysis",  
    model="gpt2"  
)
```

GPT-2 is intended for text generation. Using it directly for sentiment analysis demonstrates the importance of selecting a model appropriate for the required task.

D. Text Generation Parameters

The GPT-2 model is used for text generation:

```
generator = pipeline("text-generation", model="gpt2")
```

The following parameters are used:

```
result = generator(  
    text,  
    max_new_tokens=50,  
    temperature=0.7  
)
```

`max_new_tokens` specifies the maximum number of new tokens to generate.

`temperature` controls the randomness of the generated output.

Sentiment Analysis and Text Generation

===== MENU =====

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit

[Fix Code](#)

Enter your choice: 1

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f. Using a pipeline without specifying a model name and revision in production is not recommended.

Loading weights: 100%  104/104 [00:00<00:00, 4062.05it/s]

Enter text: I Love this Movie.

Result: [{'label': 'POSITIVE', 'score': 0.9998736381530762}]

===== MENU =====

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit

Enter your choice: 2

Loading weights: 100%  148/148 [00:00<00:00, 4633.69it/s]

Enter prompt: Explain about Artificial Intelligence.

[transformers] Both `max\_new\_tokens` (=50) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Generated Text:

Explain about Artificial Intelligence.

We use three types of artificial intelligence:

intelligent (intelligent-but-not-intelligent) or general intelligence (intelligent-but-not-general)

We develop intelligent software that can be used to make

Summarization and Question Answering

===== MENU =====

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit

Enter your choice: 3

No model was supplied, defaulted to sshleifer/distilbart-cnn-12-6 and revision a4f8f3e (<https://huggingface.co/sshleifer/distilbart-cnn-12-6>).

Using a pipeline without specifying a model name and revision in production is not recommended.

Device set to use mps:0

Enter long text:

Summarization is an NLP task used to generate a shorter and more meaningful version of a longer text while preserving its important information. It helps reduce the length of large documents, articles, reports, or paragraphs so that the main ideas can be understood quickly. In the Transformers library, the pipeline("summarization") function can be used to perform automatic text summarization using a pre-trained Transformer model. The input text is provided to the summarization pipeline, which processes the text and generates a concise summary. Parameters such as max_length and min_length can be used to control the length of the generated summary, while do_sample=False can be used to generate a deterministic summary.

Summary:

In Transformers library, pipeline can be used to perform automatic text summarization using a pre-trained Transformer model. The input text is provided to the summarization pipeline, which processes the text and generates a concise summary.

===== MENU =====

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit

Enter your choice: 4

No model was supplied, defaulted to distilbert/distilbert-base-cased-distilled-squad and revision 564e9b5 (<https://huggingface.co/distilbert/distilbert-base-cased-distilled-squad>).

Using a pipeline without specifying a model name and revision in production is not recommended.

Device set to use mps:0

Enter Context:

Delhi is capital of india.

Enter Question: what is capital of india?

Answer: Delhi

PROGRAM CODE

```
!pip install transformers==4.53.0
```

```
!pip install torch
```

```
!pip install sentencepiece
```

```
from transformers import pipeline
```

```
def sentiment_analysis():
```

```
    classifier = pipeline("sentiment-analysis")
```

```
    text = input("Enter text: ")
```

```
    result = classifier(text)
```

```
    print("\nResult:", result)
```

```
def text_generation():
```

```
    generator = pipeline("text-generation", model="gpt2")
```

```
    text = input("Enter prompt: ")
```

```
    result = generator(
```

```
        text,
```

```
        max_new_tokens=50,
```

```
        temperature=0.7
```

```
    )
```

```
    print("\nGenerated Text:")
```

```
    print(result[0]["generated_text"])
```

```
def summarization():
```

```
    summarizer = pipeline("summarization")
```

```
    text = input("Enter long text:\n")
```

```
    result = summarizer(text, max_length=50, min_length=20, do_sample=False)
```

```
    print("\nSummary:")
```

```
    print(result[0]["summary_text"])
```

```
def question_answering():
```

```
    qa = pipeline("question-answering")
```

```
    context = input("Enter Context:\n")
```

```
    question = input("Enter Question: ")
```

```
    result = qa(question=question, context=context)
```

```
    print("\nAnswer:", result["answer"])
```

Translation And Exit...

===== MENU =====

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit

Enter your choice: 5

```
/opt/anaconda3/lib/python3.13/site-packages/transformers/models/marian/tokenization_marian.py:175: UserWarning: Recommended: pip install sacremoses.
```

```
warnings.warn("Recommended: pip install sacremoses.")
```

```
Device set to use mps:0
```

Enter English text: Summarization is an NLP task used to generate a shorter and more meaningful version of a longer text while preserving its important information. It helps reduce the length of large documents, articles, reports, or paragraphs so that the main ideas can be understood quickly. In the Transformers library, the pipeline("summarization") function can be used to perform automatic text summarization using a pre-trained Transformer model. The input text is provided to the summarization pipeline, which processes the text and generates a concise summary. Parameters such as max_length and min_length can be used to control the length of the generated summary, while do_sample=False can be used to generate a deterministic summary.

Hindi Translation:

संक्षेपण एक NLP कार्य है जो लंबे पाठ के एक छोटे से और अधिक अर्थपूर्ण संस्करण तैयार करने के लिए प्रयोग किया जाता है। यह बड़े दस्तावेजों, आलेखों, रिपोर्टों, या पैराग्राफ की लंबाई को कम करने में मदद करता है ताकि मुख्य विचारों को जल्दी समझ में लिया जा सके। रूपांतरण लाइब्रेरी में, pipeline("summarization") फ़ंक्शन एक स्वचालित मॉडल उपयोग में लिया जा सकता है। आप पाठ का उपयोग करने के लिए पाठ का उपयोग किया जा सकता है, और सारांश पेश किया जा सकता है, और सारांश पेश किया जा सकता है। उदाहरण के लिए, उदाहरण के रूप में प्रयोग किया जा सकता है, उदाहरण के लिए।

===== MENU =====

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit

Enter your choice: 6

Exiting...

```
def translation():
    translator = pipeline(
        "translation",
        model="Helsinki-NLP/opus-mt-en-hi"
    )
    text = input("Enter English text: ")
    result = translator(text)
    print("\nHindi Translation:")
    print(result[0]["translation_text"])

def different_models():
    text = input("Enter text: ")

    print("\nModel 1:")
    classifier = pipeline(
        "sentiment-analysis",
        model="distilbert-base-uncased-finetuned-sst-2-english"
    )
    result = classifier(text)
    print(result)

    print("\nModel 2:")
    classifier = pipeline(
        "sentiment-analysis",
        model="lxyuan/distilbert-base-multilingual-cased-sentiments-student"
    )
    result = classifier(text)
    print(result)

def wrong_model():
    print("\nTrying GPT-2 for Sentiment Analysis")

    try:
        classifier = pipeline(
            "sentiment-analysis",
            model="gpt2"
        )

        text = input("Enter text: ")
        result = classifier(text)
```



```
print("\nResult:", result)
```

```
except Exception as e:
```

```
print("\nError:", e)
```

```
while True:
```

```
    print("\n===== MENU =====")
```

```
    print("1. Sentiment Analysis")
```

```
    print("2. Text Generation")
```

```
    print("3. Summarization")
```

```
    print("4. Question Answering")
```

```
    print("5. Translation")
```

```
    print("6. Different Models")
```

```
    print("7. Model for Another Task")
```

```
    print("8. Exit")
```

```
choice = input("Enter your choice: ")
```

```
if choice == "1":
```

```
    sentiment_analysis()
```

```
elif choice == "2":
```

```
    text_generation()
```

```
elif choice == "3":
```

```
    summarization()
```

```
elif choice == "4":
```

```
    question_answering()
```

```
elif choice == "5":
```

```
    translation()
```

```
elif choice == "6":
```

```
    different_models()
```

```
elif choice == "7":
```

```
    wrong_model()
```

```
elif choice == "8":
```

```
    print("Exiting...")
```

```
    break
```

```
else:
```

```
    print("Invalid choice!")
```


RESULT

The menu-driven NLP application was successfully implemented using the Transformers pipeline() function.

The application was able to perform:

- 1.Sentiment Analysis
- 2.Text Generation
- 3.Summarization
- 4.Question Answering
- 5.English-to-Hindi Translation

Different models were also tested using the model parameter. A model intended for another task was tested, and GPT-2 text generation was performed using max_new_tokens and temperature.

Output: The respective result is displayed according to the option selected by the user.

LEARNING OUTCOMES

1. Learned how to use the Transformers pipeline() function.
2. Learned how to perform multiple NLP tasks using pre-trained models.
3. Learned how to create a menu-driven NLP application.
4. Learned how pipeline() can automatically select suitable models.
5. Learned how to specify a particular model using the model parameter.
6. Learned how different models can be used for the same NLP task.
7. Understood why a model designed for one task may not work correctly for another task.
8. Learned how max_new_tokens and temperature affect text generation.
9. Gained practical experience with Transformer-based NLP applications.

EXPERIMENT – 2

PROBLEM STATEMENT: Take one/two functions from Experiment 1, such as Sentiment Analysis, and create a Gradio application out of it. The Gradio application has to be deployed on Render / Hugging Face.

OBJECTIVES

1. To create a web-based interface for the Sentiment Analysis function developed in Experiment 1.
2. To understand the basic working of the Gradio library.
3. To connect a Transformer-based machine learning model with a Gradio interface.
4. To accept text input from the user through a web interface.
5. To display the predicted sentiment through the Gradio application.
6. To deploy the Gradio application on a cloud platform such as Render or Hugging Face.

THEORY

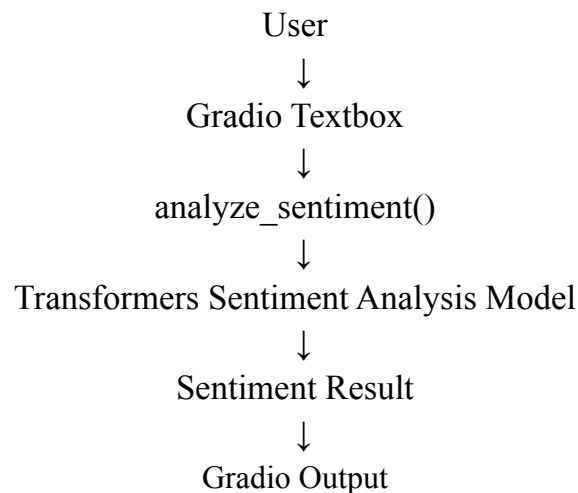
1. Overview of the Experiment

Gradio is a Python library used to create simple web interfaces for machine learning models and Python functions.

In this experiment, the Sentiment Analysis function from Experiment 1 is converted into a web application using Gradio.

The user enters text into a textbox. The text is passed to the Transformer sentiment-analysis pipeline. The model analyzes the text and returns its predicted sentiment.

The basic flow of the application is:



2. Libraries Used

Gradio

Gradio is used to create the web interface for the sentiment analysis application.

`import gradio as gr`

Transformers

The Transformers library is used to load the pre-trained sentiment analysis model through the pipeline() function.

`from transformers import pipeline`

3. Functions Used

pipeline()

The pipeline() function creates a sentiment-analysis pipeline.

```
classifier = pipeline("sentiment-analysis")
```

analyze_sentiment()

This user-defined function receives the text entered by the user, passes it to the classifier, and returns the predicted sentiment.

```
def analyze_sentiment(text):  
    result = classifier(text)  
    return result[0]["label"]
```

gr.Interface()

gr.Interface() connects the Python function with the input and output components of the web interface.

```
app = gr.Interface(  
    fn=analyze_sentiment,  
    inputs=gr.Textbox(label="Enter the Sentiment"),  
    outputs=gr.Textbox(label="Analyzed Sentiment result")  
)
```

app.launch()

The launch() function starts the Gradio application.

```
app.launch(share=True)
```

The share=True option creates a shareable link for the running Gradio application.

WORKING OF PROGRAM

- The program first imports Gradio and the Transformers pipeline() function.

```
import gradio as gr
```

```
from transformers import pipeline
```

- A sentiment-analysis classifier is created:

```
classifier = pipeline("sentiment-analysis")
```

- The analyze_sentiment() function receives the text entered by the user.

```
def analyze_sentiment(text):  
    if len(text) == 0:  
        return "Please enter some text"
```

```
    result = classifier(text)
```

```
    return result[0]["label"]
```

If the user does not enter any text, the application displays a message asking the user to enter text. Otherwise, the text is passed to the sentiment-analysis model.

- The predicted label is extracted using:

```
result[0]["label"]
```


- The Gradio interface is then created:

```
app = gr.Interface(  
    fn=analyze_sentiment,  
    inputs=gr.Textbox(label="Enter the Sentiment"),  
    outputs=gr.Textbox(label="Analyzed Sentiment result")  
)
```

- Finally, the application is launched:

```
app.launch(share=True)
```

PROGRAM CODE

```
import gradio as gr  
from transformers import pipeline
```

```
classifier = pipeline("sentiment-analysis")
```

```
def analyze_sentiment(text):
```

```
    if len(text) == 0:  
        return "Please enter some text"
```

```
    result = classifier(text)  
    return result[0]["label"]
```

```
app = gr.Interface(  
    fn=analyze_sentiment,  
    inputs=gr.Textbox(label="Enter the Sentiment"),  
    outputs=gr.Textbox(label="Analyzed Sentiment result")  
)
```

```
app.launch(share=True)
```

DEPLOYMENT

1. The Gradio application is required to be deployed on Render or Hugging Face as specified in the experiment.
2. For deployment, the project should contain the Python application and its required dependencies.
3. A requirements.txt file can contain:
 - gradio
 - transformers
 - torch
 - sentencepiece
4. The application file contains the Gradio code and is executed by the deployment platform.
5. After successful deployment, the application can be accessed through the URL provided by the hosting platform.

Output: Gradio based application

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not recommended.

Loading weights: 100%  104/104 [00:00<00:00, 3752.74it/s]

* Running on local URL: <http://127.0.0.1:7862>

* Running on public URL: <https://6d6a3b0cce4e3dfcbc.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (<https://huggingface.co/spaces>)

Enter the Sentiment good movie	Analyzed Sentiment result POSITIVE	
Clear	Submit	Flag

RESULT

A Gradio-based Sentiment Analysis web application was successfully created using the Sentiment Analysis function from Experiment 1.

The application accepts text from the user and displays the predicted sentiment as the output.

Example:

Input:

I really enjoyed this movie.

Output:

POSITIVE

The application can also be deployed on Render or Hugging Face according to the experiment requirement.

LEARNING OUTCOMES

After completing this experiment:

1. Learned how to create a simple web interface using Gradio.
2. Learned how to connect a Transformer model with a Gradio application.
3. Learned how to accept user input through a web interface.
4. Learned how to display machine learning predictions through Gradio.
5. Learned how to reuse the Sentiment Analysis function from Experiment 1.
6. Learned the basic process of deploying a machine learning application on a cloud platform.
7. Understood how a Python-based machine learning function can be converted into an interactive web application.

EXPERIMENT – 3

PROBLEM STATEMENT

Perform the following Natural Language Processing tasks:

- 1.Perform Tokenization using spaCy (Classical NLP).
- 2.Perform Tokenization using tokenizers of popular modern NLP models and compare the tokenization outputs of BERT, GPT, T5, and DeepSeek on the same input.
- 3.Perform Tokenization using spaCy after reading text from a PDF file and a DOCX file.
- 4.Write a program to take a paragraph as input and remove the stopwords using spaCy.
- 5.Test different attributes of a spaCy Token including lower_, shape_, prefix_, suffix_, is_alpha, is_digit, is_numeric, is_punct, is_space, is_title, like_email, like_url, i, idx, n_lefts, and n_rights.

OBJECTIVES

- 1.To understand tokenization in Natural Language Processing.
- 2.To perform classical NLP tokenization using spaCy.
- 3.To understand how modern Transformer models tokenize text.
- 4.To compare the tokenization behavior of BERT, GPT-2, T5, and DeepSeek.
- 5.To extract text from PDF and DOCX files and perform tokenization.
- 6.To understand and remove stopwords using spaCy.
- 7.To study different attributes of spaCy Token objects.
- 8.To understand the difference between classical NLP tokenization and modern Transformer tokenization.

THEORY

1. Overview of the Experiment

Tokenization

Tokenization is the process of breaking text into smaller units called **tokens**. A token can be a word, punctuation mark, number, or a subword depending on the tokenizer being used.

For example:

Input:Natural Language Processing is amazing!

Output:

Natural
Language
Processing
is
amazing
!

Tokenization is an important preprocessing step in NLP because machine learning models process text through numerical representations of tokens.

spaCy Tokenization

spaCy is a Python library used for Natural Language Processing. Its tokenizer divides text into tokens and provides several useful linguistic attributes for each token.

Example:

```
doc = nlp(text)
for token in doc:
    print(token.text)
```

Transformer Tokenization

Modern Transformer models use their own tokenization methods. Different models can divide the same sentence into different tokens.

In this experiment, tokenization is compared for:

- BERT
- GPT-2
- T5
- DeepSeek

The AutoTokenizer class from Transformers is used to load the appropriate tokenizer for each model.

Stopwords

Stopwords are commonly occurring words that may carry less useful information for some NLP tasks.

Examples include:

the
is
a
an
and
of
to

spaCy provides the `is_stop` attribute to identify stopwords.

spaCy Token Attributes

spaCy provides many attributes that describe individual tokens.

`lower_`: Lowercase form of the token

`shape_`: Shape pattern of the token

`prefix_`: Prefix of the token

`suffix_`: Suffix of the token

`is_alpha`: Checks whether the token contains alphabetic characters

`is_digit`: Checks whether the token consists of digits

`is_numeric`: Checks whether the token represents a number

`is_punct`: Checks whether the token is punctuation

`is_space`: Checks whether the token is whitespace

`is_title`: Checks whether the token is title-cased

`like_email`: Checks whether the token resembles an email address

like_url: Checks whether the token resembles a URL

i: Position/index of the token in the document

idx: Character offset of the token in the original text

n_lefts: Number of left dependency children

n_rights: Number of right dependency children

These are the attributes specified in the experiment.

2. LIBRARIES USED

spaCy

Used for classical NLP tokenization, stopwords removal, and testing token attributes.

```
import spacy
```

Transformers

Used for loading the tokenizers of BERT, GPT-2, T5, and DeepSeek.

```
from transformers import AutoTokenizer
```

PyPDF

Used to read and extract text from PDF files.

```
from pypdf import PdfReader
```

python-docx

Used to read and extract text from DOCX files.

```
from docx import Document
```

The required libraries can be installed using:

```
!pip install spacy transformers torch pypdf python-docx
```

```
!python -m spacy download en_core_web_sm
```

3. FUNCTIONS USED

spacy.load()

Loads the spaCy English language model.

```
nlp = spacy.load("en_core_web_sm")
```

nlp()

Processes input text and creates a spaCy Doc object.

```
doc = nlp(text)
```

AutoTokenizer.from_pretrained()

Loads the tokenizer associated with a particular Transformer model.

```
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

tokenize()

Converts text into tokens using the selected Transformer tokenizer.

```
tokens = tokenizer.tokenize(text)
```


convert_tokens_to_ids()

Converts the generated tokens into their corresponding numerical token IDs.

```
tokenizer.convert_tokens_to_ids(tokens)
```

PdfReader()

Reads a PDF document.

```
reader = PdfReader("sample.pdf")
```

Document()

Reads a DOCX document.

```
document = Document("sample.docx")
```

input()

Accepts a paragraph from the user.

print()

Displays tokens, extracted text, and token attributes.

WORKING OF PROGRAM

The experiment is divided into five parts.

(i) Tokenization using spaCy

First, the English spaCy model is loaded:

```
nlp = spacy.load("en_core_web_sm")
```

The input text is processed:

```
doc = nlp(text)
```

The program then iterates through the document and prints every token.

for token in doc:

```
    print(token.text)
```

(ii) Tokenization using BERT, GPT-2, T5 and DeepSeek

The AutoTokenizer class is used to load the tokenizer for each model.

The models used are:

```
models = {  
    "BERT": "bert-base-uncased",  
    "GPT-2": "gpt2",  
    "T5": "t5-small",  
    "DeepSeek": "deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B"  
}
```

The same input text is passed to each tokenizer.

```
tokens = tokenizer.tokenize(text)
```

The tokens and their corresponding token IDs are displayed.

This allows the tokenization outputs of different modern NLP models to be compared.

(iii) Tokenization from PDF and DOCX

A PDF file is opened using:

```
reader = PdfReader("sample.pdf")
```

Text is extracted from each page.

Similarly, the DOCX file is opened using:

```
document = Document("sample.docx")
```

Text is extracted from each paragraph.

The extracted text from both files is then passed to spaCy:

```
doc = nlp(text)
```

and:

```
docx_doc = nlp(docx_text)
```

The tokens are then printed.

(iv) Removing Stopwords

The user enters a paragraph:

```
text = input("Enter a paragraph: ")
```

The paragraph is processed using spaCy.

Each token is checked using:

```
if not token.is_stop:
```

Only tokens that are not stopwords are added to the result.

The program then prints the paragraph after removing stopwords.

The program also demonstrates removal of both stopwords and punctuation:

```
if not token.is_stop and not token.is_punct:
```

(v) Testing spaCy Token Attributes

A sample sentence containing normal words, a URL, an email address, and a number is processed.

For every token, the program displays the required attributes.

The missing `is_numeric` attribute has been added:

```
print("is_numeric :", token.is_numeric)
```

Tokenization using spaCy

Original Text:
Natural Language Processing is an important field of Artificial Intelligence.

Tokens:
Natural
Language
Processing
is
an
important
field
of
Artificial
Intelligence
.

Tokenization using BERT, GPT-2, T5 and DeepSeek

=====

BERT

=====

Tokens:

['natural', 'language', 'processing', 'is', 'amazing', '!']

Token IDs:

[3019, 2653, 6364, 2003, 6429, 999]

=====

GPT-2

=====

Tokens:

['Natural', 'GLanguage', 'GProcessing', 'Gis', 'Gamazing', '!']

Token IDs:

[35364, 15417, 28403, 318, 4998, 0]

=====

T5

=====

Tokens:

['_Natural', '_Language', '_Processing', '_is', '_amazing', '!']

Token IDs:

[6869, 10509, 19125, 19, 1237, 55]

=====

DeepSeek

=====

Tokens:

['Natural', 'GLanguage', 'GProcessing', 'Gis', 'Gamazing', '!']

Token IDs:

[54281, 11434, 28125, 374, 7897, 0]

PROGRAM CODE

```
!pip install spacy transformers torch pypdf python-docx
!python -m spacy download en_core_web_sm
```

```
# (i) Tokenization using spaCy
```

```
import spacy
nlp = spacy.load("en_core_web_sm")

text = "Natural Language Processing is an important field of Artificial Intelligence."
doc = nlp(text)

print("Original Text:")
print(text)

print("\nTokens:")
for token in doc:
    print(token.text)
```

```
#(ii) Tokenization using BERT, GPT, T5 and DeepSeek
```

```
from transformers import AutoTokenizer
text = "Natural Language Processing is amazing!"

models = {
    "BERT": "bert-base-uncased",
    "GPT-2": "gpt2",
    "T5": "t5-small",
    "DeepSeek": "deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B"
}

for model_name, model_id in models.items():
    print("\n" + "=" * 50)
    print(model_name)
    print("=" * 50)

    tokenizer = AutoTokenizer.from_pretrained(model_id)
    tokens = tokenizer.tokenize(text)
    print("Tokens:")
    print(tokens)
    print("\nToken IDs:")
    print(tokenizer.convert_tokens_to_ids(tokens))
```

Tokenization from PDF and DOCX

Ignoring wrong pointing object 12 0 (offset 0)

Ignoring wrong pointing object 13 0 (offset 0)

Extracted Text from PDF:

Lorem ipsum

Vestibulum neque massa, scelerisque sit amet ligula eu, congue molestie mi. Praesent ut varius sem.,

Tokens from PDF:

Lorem

ipsum

Vestibulum

neque

massa

,

scelerisque

sit

amet

ligula

eu

,

congue

molestie

mi

.

Praesent

ut

varius

sem

.

,

Extracted Text from word:

Lorem ipsum ,Vestibulum neque massa, scelerisque sit amet ligula eu, congue molestie mi. Praesent ut varius sem.,

Tokens from Word:

Lorem

ipsum

,

Vestibulum

neque

massa

,

scelerisque

sit

amet

ligula

eu

,

congue

molestie

mi

.

Praesent

ut

varius

sem

.

,

(iii) Extract and read text from PDF and DOCX

```
import spacy
from pypdf import PdfReader
from docx import Document

nlp = spacy.load("en_core_web_sm")

reader = PdfReader("sample.pdf")
document = Document("sample.docx")

text = ""
docx_text = ""
for page in reader.pages:
    page_text = page.extract_text()

    if page_text:
        text += page_text + "\n"

print("Extracted Text from PDF:")
print(text)
doc = nlp(text)
print("\nTokens from PDF:")
for token in doc:
    print(token.text)

for paragraph in document.paragraphs:
    docx_text += paragraph.text + "\n"

print("Extracted Text from Word:")
print(docx_text)

docx_doc = nlp(docx_text)
print("\nTokens from Word:")
for token in docx_doc:
    print(token.text)

#(iv) Remove Stopwords using spaCy

import spacy
nlp = spacy.load("en_core_web_sm")
text = input("Enter a paragraph: ")
doc = nlp(text)
```



```
print("\nOriginal Paragraph:")
print(text)
print("\nAfter Removing Stopwords:")
result = []
```

```
for token in doc:
    if not token.is_stop:
        result.append(token.text)
```

```
print(" ".join(result))
```

```
# Remove punctuation along with stopwords
```

```
import spacy
nlp = spacy.load("en_core_web_sm")
text = input("Enter a paragraph: ")
doc = nlp(text)
result = []
for token in doc:
    if not token.is_stop and not token.is_punct:
        result.append(token.text)
```

```
print("\nAfter Removing Stopwords and Punctuation:")
print(" ".join(result))
```

```
 #(v) Test spaCy Token Attributes
```

```
import spacy
nlp = spacy.load("en_core_web_sm")
text = "Hello Keshav! Visit https://example.com or email me at test@example.com. I have 25 apples."
doc = nlp(text)
for token in doc:
    print("=" * 50)
    print("Token      :", token.text)
    print("lower_     :", token.lower_)
    print("shape_      :", token.shape_)
    print("prefix_     :", token.prefix_)
    print("suffix_     :", token.suffix_)

    print("is_alpha    :", token.is_alpha)
    print("is_digit    :", token.is_digit)
    print("is_numeric  :", token.is_numeric)
    print("is_punct    :", token.is_punct)
    print("is_space    :", token.is_space)
    print("is_title    :", token.is_title)
```

Test spaCy Token Attributes

```
=====
Token      : Hello
lower_     : hello
shape_     : Xxxxx
prefix_    : H
suffix_    : llo
is_alpha   : True
is_digit   : False
is_punct   : False
is_space   : False
is_title   : True
like_email : False
like_url   : False
i          : 0
idx        : 0
n_lefts    : 0
n_rights   : 0
=====
Token      : Keshav
lower_     : keshav
shape_     : Xxxxx
prefix_    : K
suffix_    : hav
is_alpha   : True
is_digit   : False
is_punct   : False
is_space   : False
is_title   : True
like_email : False
like_url   : False
i          : 1
idx        : 6
n_lefts    : 1
n_rights   : 1
=====
Token      : !
lower_     : !
shape_     : !
prefix_    : !
suffix_    : !
is_alpha   : False
is_digit   : False
is_punct   : True
is_space   : False
is_title   : False
like_email : False
like_url   : False
i          : 2
idx        : 12
n_lefts    : 0
n_rights   : 0
=====
```

```
print("like_email :", token.like_email)
print("like_url   :", token.like_url)

print("i          :", token.i)
print("idx        :", token.idx)

print("n_lefts    :", token.n_lefts)
print("n_rights   :", token.n_rights)
```

RESULT

The following NLP operations were successfully implemented:

1. Tokenization using spaCy was performed successfully.
2. Tokenization using BERT, GPT-2, T5, and DeepSeek tokenizers was performed and compared.
3. Text was successfully extracted from PDF and DOCX files and tokenized using spaCy.
4. Stopwords were successfully removed from a user-provided paragraph.
5. Stopwords and punctuation were also removed from the paragraph.
6. The required spaCy Token attributes were successfully tested, including `is_numeric`.

The tokenization output and token IDs were displayed for the selected Transformer models.

LEARNING OUTCOMES

After completing this experiment:

- 1.Learned the concept of tokenization in NLP.
- 2.Learned how to perform tokenization using spaCy.
- 3.Learned how Transformer models use different tokenizers.
- 4.Compared tokenization outputs of BERT, GPT-2, T5, and DeepSeek.
- 5.Learned how to extract text from PDF and DOCX files.
- 6.Learned how to tokenize extracted document text using spaCy.
- 7.Learned how to identify and remove stopwords.
- 8.Learned how to remove punctuation along with stopwords.
- 9.Learned about different attributes available in a spaCy Token.
- 10.Understood how token properties such as `is_alpha`, `is_digit`, `is_numeric`, `like_email`, `like_url`, `idx`, `n_lefts`, and `n_rights` can be examined.